



验证集Loss与准确率差异分析

问题描述

验证集显示loss较大，但memory中test的CSV预测结果大部分都命中。

核心原因分析

1. Loss计算方式（基于概率分布）

训练时的Loss计算：

```
def record_eval(env):
    train_losses.append(env.evaluation_result_list[0][2])
    val_losses.append(env.evaluation_result_list[1][2])
    if callback:
        callback(len(train_losses), train_losses[-1], val_losses[-1])
```

使用的指标配置：

```

params = {
    'objective': 'multiclass',
    'num_class': 4,
    'metric': 'multi_logloss',
    'boosting_type': 'gbdt',
    'num_leaves': self.num_leaves_var.get(),
    'learning_rate': self.lr_var.get(),
    'feature_fraction': self.feature_frac_var.get(),
    'bagging_fraction': self.bagging_frac_var.get(),
    'bagging_freq': 5,
    'max_depth': self.max_depth_var.get(),
    'min_child_samples': self.min_child_var.get(),
    'verbose': -1,
    'num_boost_round': self.num_boost_var.get(),
    'early_stopping_rounds': self.early_stop_var.get()
}

```

- **使用的指标**: `multi_logloss` (第720行)
- **计算基础**: 基于模型输出的**概率分布**, 不是最终分类结果
- **公式**: $-\log(P(y_{\text{true}} | x))$, 其中P是模型对真实类别的预测概率

关键点:

- 即使预测类别正确, 如果概率不够"确定" (如[0.3, 0.3, 0.3, 0.1]), `loss`也会较大
- `Loss`衡量的是概率分布的"置信度", 不是分类准确性
- 例如: 真实标签是1, 模型输出概率[0.3, 0.35, 0.25, 0.1], 虽然`argmax=1`预测正确, 但`loss = -log(0.35) ≈ 1.05`, 仍然较大

2. 准确率计算方式 (基于分类结果)

验证集评估流程:

```

y_val_pred = self.predictor.predict(X_val)
y_val_pred_corrected = self.predictor.apply_rules(val_df, y_val_pred)
val_results = self.predictor.evaluate(y_val, y_val_pred, y_val_pred_corrected)

```

`predict`方法:

```
def predict(self, X: np.ndarray) -> np.ndarray:
    """预测"""
    y_pred_proba = self.model.predict(X, num_iteration=self.model.best_iteration)
    return np.argmax(y_pred_proba, axis=1)
```

- **准确率计算**：使用 `accuracy_score`，基于**最终分类结果**
- **判断标准**：只要 `argmax` 的结果正确，就算命中
- **关键差异**：准确率只看类别是否正确，不关心概率值的大小

3. 规则修正的影响（关键因素）

`apply_rules`方法应用了两个重要规则：

```

def apply_rules(self, df: pd.DataFrame, predictions: np.ndarray) -> np.ndarray:
    """
    应用后处理规则：
    1. 当一口井最大状态是2的时候，即使预测的是3，也当作2处理（仅当有真实标签时）
    2. 确保状态转换的单向性（0 -> 1 -> 2 -> 3）
    """
    print("\n应用后处理规则...")

    # 创建副本以避免修改原始数据
    temp_df = df.copy()

    # 必须确保temp_df和predictions是对齐的
    # 添加一个0..N的索引列，用于后续将结果放回数组的正确位置
    temp_df['__row_idx__'] = range(len(temp_df))
    temp_df['__pred__'] = predictions

    corrected_predictions = predictions.copy()

    # 按井号分组处理
    for well_name, group in temp_df.groupby('转换后JH'):
        # 获取该组在原始数组中的索引位置
        indices = group['__row_idx__'].values
        preds = group['__pred__'].values.copy()

        # 规则1：如果真实最大状态为2，则将预测的3改为2
        # 只有当DataFrame包含'status'列时才能应用此规则
        if 'status' in group.columns:
            max_status = group['status'].max()
            if max_status == 2:
                preds[preds == 3] = 2

        # 规则2：强制单向性 0 -> 1 -> 2 -> 3
        corrected = []
        current_max = 0

        for pred in preds:
            if pred >= current_max:
                current_max = pred
            corrected.append(current_max)

```

```

        # 将修正后的结果放回对应位置
        corrected_predictions[indices] = corrected

    # 统计修正的数量
    n_corrected = np.sum(predictions != corrected_predictions)
    print(f"修正了 {n_corrected} 个预测 ({n_corrected/len(predictions)*100:.2f}%)")

    return corrected_predictions

```

规则2（强制单向性）的作用：

- 强制状态只能递增，不能回退
- 如果真实标签也遵循这个规律（井段状态通常是单向发展的），修正后的准确率会显著提高
- **这是导致准确率高但loss大的主要原因**

4. 保存到memory的结果

保存的是修正后的结果：

```

self.predictor.save_results_by_well(train_df, y_train_pred_corrected, os.path.join(memory_dir, "train"))
self.predictor.save_results_by_well(val_df, y_val_pred_corrected, os.path.join(memory_dir, "test"))

```

- 保存的是 `y_val_pred_corrected`，不是原始预测 `y_val_pred`
- 经过规则修正后，准确率已经大幅提升
- **这就是为什么memory中test的CSV预测结果大部分都命中的原因**

具体示例说明

假设一个井段的状态序列（7个样本）：

场景1：概率不确定但预测正确

- **真实标签**：[0, 0, 1, 1, 2, 2, 2]
- **模型概率输出**（每个样本4个类别的概率）：
 - 样本1: [0.35, 0.30, 0.20, 0.15] → argmax=0 ✓
 - 样本2: [0.32, 0.33, 0.20, 0.15] → argmax=1 ✗（真实是0）

- 样本3: [0.25, 0.40, 0.25, 0.10] → $\text{argmax}=1$ ✓
- 样本4: [0.30, 0.35, 0.25, 0.10] → $\text{argmax}=1$ ✓
- 样本5: [0.20, 0.30, 0.35, 0.15] → $\text{argmax}=2$ ✓
- 样本6: [0.15, 0.25, 0.30, 0.30] → $\text{argmax}=2$ 或3 (概率接近)
- 样本7: [0.10, 0.20, 0.35, 0.35] → $\text{argmax}=2$ 或3 (概率接近)
- **原始预测**: [0, 1, 1, 1, 2, 3, 3]
 - 准确率: $4/7 \approx 57\%$
 - Loss: 较大 (因为概率不够确定, 如样本2的真实类别0的概率只有0.32)
- **规则修正后**: [0, 1, 1, 1, 2, 2, 2] (强制单向性)
 - 修正后准确率: $6/7 \approx 86\%$
 - **准确率大幅提升!**

场景2: 状态回退被修正

- **真实标签**: [0, 0, 1, 1, 2, 2, 2]
- **模型原始预测**: [0, 0, 1, 0, 2, 2, 2] (样本4预测回退到0)
- **规则修正后**: [0, 0, 1, 1, 2, 2, 2] (强制不能回退)
 - 修正后准确率: $7/7 = 100\%$

总结

为什么loss大但准确率高?

1. **Loss基于概率分布**: 即使预测正确, 概率不够确定也会导致loss大
2. **准确率基于分类结果**: 只要 argmax 正确就算命中
3. **规则修正提升准确率**: 单向性规则修正了很多预测错误, 特别是状态回退的情况
4. **保存的是修正结果**: memory中test保存的是修正后的结果, 准确率已经提升

总结

为什么loss大但准确率高?

1. **Loss基于概率分布**: 即使预测正确, 概率不够确定也会导致loss大
2. **准确率基于分类结果**: 只要 argmax 正确就算命中
3. **规则修正提升准确率**: 单向性规则修正了很多预测错误, 特别是状态回退的情况

4. **保存的是修正结果**：memory中test保存的是修正后的结果，准确率已经提升

关键发现：

- 训练时显示的loss是基于**概率分布**的 `multi_logloss`
- 验证集评估的准确率是基于**分类结果**的 `accuracy_score`
- 保存到memory的是**规则修正后**的结果，准确率已经大幅提升
- 这是正常现象，loss和准确率衡量的是不同的方面

建议：

- 如果关注准确率，应该看修正后的准确率（这是实际应用的效果）
- 如果关注模型置信度，应该看loss值（反映模型对预测的确定性）
- 可以考虑在训练时也使用准确率作为评估指标，或者使用加权loss
- 可以在训练时同时监控loss和准确率，更全面地评估模型性能